INTERNSHIP PROJECT REPORT

EduGenie: Google Gemini-Powered Learning Assistant

An AI-Powered Web Application for Personalized Explanations, Interactive Tutoring, and Smart Quizzes

Domain	Google Gemini Generative AI
Team ID	xxxxxxx
Team Size	1
Submitted By	Manan Kochar (Team Lead)
Repository	https://github.com/manankochar1720/Gen-ai-Google-Gemini-Powered-Learning-Assistant

Submitted in partial fulfillment of the requirements for the
Internship Program — 2026

1. INTRODUCTION

1.1 Project Overview

EduGenie: Google Gemini-Powered Learning Assistant is an intelligent, AI-driven educational portal designed to transform how students understand academic concepts, practice quizzes, and interact with private tutors. Leveraging generative AI models, the application provides instantaneous, high-quality, level-specific explanations along with interactive mock quizzes and full-length PDF scorecards, all encapsulated in a modern dashboard interface.

The system architecture utilizes a high-performance Python stack: Streamlit for the presentation tier, FastAPI for the backend API routing system, and the Google Gemini Pro API for contextual LLM analysis. Together, these technologies deliver a robust, highly responsive learning platform that supports personalized educational journeys.

Core Usage Scenarios

• **Scenario 1 - Topic Explainer:** A student inputs a topic (e.g. 'Calculus Limits') and selects their target expertise level (Beginner/Intermediate/Advanced) and delivery style (Technical, Analogy-based, or Visual). The platform generates tailored explanations.

• **Scenario 2 - Chat Tutor:** An interactive chat assistant maintains local conversation history, allowing students to ask follow-up queries and receive contextual responses about their study topics.

• **Scenario 3 - Quiz Generator:** Students test their retention through multiple-choice questions automatically generated by Gemini, completed with instant grading, scoreboard history logging, and PDF report downloads.

1.2 Purpose

The purpose of this project is to address the limitations of generic, static textbooks by providing an interactive, responsive, and private self-paced learning portal. It provides clear, analogy-based explanations that scale according to student expertise, reducing educational costs while keeping lessons engaging and student-centered.

2. IDEATION PHASE

2.1 Problem Statement

Understanding complex academic concepts is often limited by dry textbooks and rigid curriculums. Using the customer problem statement framework (I am / I'm trying to / But / Because / Which makes me feel), the core issues faced by modern self-paced learners were identified:

I am	I'm trying to	But	Because	Which makes me feel
A student struggling with complex subjects.	Understand core physics and math theories.	Online guides are too dense or generic.	There is no teacher to explain it via analogies.	**Confused and demotivated.**
An exam candidate testing my retention.	Practice customized topic-specific quizzes.	Pre-made tests are paywalled or generic.	I cannot generate fresh quizzes on specific topics.	**Unprepared and stressed.**

2.2 Empathy Map Canvas

An empathy map canvas was designed to capture the target audience's study mindset and their emotional hurdles when self-studying:

Aspect	Description
Think & Feel	Anxious about exams; wants a tutor who explains topics simply without judging.
Hear	Lectures that move too fast; peer discussions about competitive exam preparations.
See	Piles of expensive textbooks; random YouTube videos that lack structured quiz practices.
Say & Do	Admits they are lost; searches online forums for analogies and visual explanations.
Pain	Wasting hours reading dry content; losing interest in STEM subjects.
Gain	A clear, personalized tutor that matches their level, offering instant self-testing.

2.3 Brainstorming

A structured ideation and brainstorming session was conducted to explore possible solution directions before converging on the final concept.

Step 1: Problem Selection

Conventional learning pathways fail because they are not personalized. Standard video tutorials or blogs offer no feedback loop. The selection focus converged on creating a self-service interactive portal that merges explanation customizability with instant validation.

Step 2: Idea Listing & Grouping

Candidate ideas were mapped into five core functional areas: (1) Customized Explanation Engine, (2) Conversational Tutor Assistant, (3) JSON Schema-based Quiz Engine, (4) Persistent Local JSON History Logs, and (5) PDF Report Card Generator. A modular web dashboard (Streamlit) combined with a robust API framework (FastAPI) was selected.

Step 3: Idea Prioritization

Using an Importance vs. Feasibility grid, the Explainer Engine, Chat interface, and Quiz Generator scored highest on both feasibility and user value. Secondary scope enhancements, such as cloud Database sync, voice-enabled chat, and class-sharing integrations were cataloged as future roadmap items.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The customer journey details the user's touchpoints, actions, thoughts, and emotional response across three distinct learning stages:

Stage 1: Discovery & Entry

• **Actions:** User enters a topic they need help with, selecting their difficulty level and explanation style.

• **Touchpoints:** Dashboard search panel, level selector dropdowns, style buttons.

• **Thoughts:** 'Will this AI explanation actually be simpler than my textbook?' — hopeful.

Stage 2: Active Study & Chat

• **Actions:** User reads the generated explanation and opens the sidebar to ask follow-up questions to the AI tutor.

• **Touchpoints:** Explanation display card, interactive Chat Drawer interface.

• **Thoughts:** 'Awesome, the analogy makes sense. Let me ask why this variable exists.' — engaged.

Stage 3: Testing & Reports

• **Actions:** User clicks 'Generate Quiz', submits their answers, gets their scorecard, and downloads a PDF summary.

• **Touchpoints:** Quiz form, score summary cards, PDF download link.

• **Thoughts:** 'I got 3/3 right! Let me download this scorecard to track my progress.' — satisfied.

3.2 Solution Requirement - Functional Requirements

FR No.	Requirement	Sub-Requirement Details
FR-1	Custom Explainer Engine	Pass topic, level, and style to FastAPI; query Gemini; display formatted explanation.
FR-2	AI Chat Assistant	Maintain chat history inside sidebar drawer; generate contextual tutor replies.
FR-3	Adaptive Quiz Engine	Query Gemini to generate MCQs matching the topic and complexity level.
FR-4	Instant Scorecard	Grade submitted options immediately; display scores and explanation of answers.
FR-5	PDF Performance Report	Generate downloadable ReportLab PDF containing topic performance metrics.

Non-Functional Requirements

NFR No.	Requirement	Description / Metrics
NFR-1	Usability	Responsive, minimal, and premium layout using Streamlit. Clean navigation panels.
NFR-2	Security	API keys are isolated in backend config.py, preventing exposure to client.
NFR-3	Reliability	Graceful fallbacks and UI notifications for network drops or API timeouts.
NFR-4	Performance	Fast API response times (typically under 2 seconds) by using Gemini Flash model.
NFR-5	Portability	Runs seamlessly across Windows, macOS, and Linux platforms using Python.

3.3 Data Flow Diagram (DFD)

The Data Flow Diagram describes the transaction flow across the presentation, application, and integration tiers:

DFD Level 0 Flow:
1. User Inputs topic/settings -> 2. Streamlit Web Client captures inputs -> 3. HTTP POST request triggered -> 4. FastAPI Backend router processes payload -> 5. Gemini Service builds schema prompt -> 6. Google Gemini API generates structured JSON response -> 7. Response parsed by Backend -> 8. JSON payload returned to Client -> 9. Dashboard renders tables/charts -> 10. Local JSON history logs updated.

User Stories - Onboarding & Scopes

ID	User Type	Epic Focus	User Story / Core Task	Priority
US-1	Student	Workspace Scaffolding	Access dashboard interface to search topics and read customized contents.	High

User Stories (Continued)

ID	User Type	Epic Focus	User Story / Core Task	Priority
US-2	Struggling Student	Custom Explainer	Request topic explanations tailored with real-world analogies.	High
US-3	Self-Learner	AI Chat Assistant	Ask follow-up questions inside a persistent chat sidebar drawer.	High
US-4	Exam Candidate	Quiz Engine	Generate customizable MCQs to test retention and get instant scoring.	High
US-5	Student	Performance PDF	Export study summaries and quiz scores to print/share with peers.	Medium
US-6	Tutor	History Tracker	Review previous study topics and quiz score history cards.	Medium

3.4 Technology Stack Selection

Selecting the right combination of tools ensures modularity, high processing speed, and alignment with Generative AI capabilities. The table below lists the core components chosen:

#	Component / Layer	Technology Chosen	Justification Details
1	**Frontend UI**	Streamlit	Rapid development of interactive widgets, seamless sidebar chat component integrations.
2	**State Management**	Session State	Tracks current topic, chat history, and active quiz details across re-runs.
3	**Backend API Router**	FastAPI	High-performance async request routing, automated interactive Swagger UI docs.

Technology Stack (Continued)

#	Component / Layer	Technology Chosen	Justification Details
4	**Generative AI Model**	Google Gemini API (1.5 Flash)	Extremely low latency, native structured JSON outputs, robust contextual chat logic.
5	**Database & Logs**	Local JSON Storage	Zero-overhead structured data persistence for study logs, feedback, and history.
6	**PDF Generator**	ReportLab	Programmatic, dynamic creation of A4 PDF performance scorecards and study summaries.

Stack Architecture Advantages

• **Decoupled Development:** The FastAPI backend acts as a central hub, allowing alternative frontends (React/Vue) or mobile apps to interface with the exact same Gemini service layer in the future.
• **Fast Prototyping:** Streamlit allows the presentation layer to be written entirely in Python, matching the backend data models directly and reducing context switching.
• **Cost Efficiency:** Local JSON storage and Gemini 1.5 Flash's large free tier minimize the hosting and runtime costs of the system during development and testing phases.

4. PROJECT DESIGN

4.1 Problem – Solution Fit

The Problem-Solution Fit canvas validates that the platform directly answers real customer needs and solves the primary pain points of study management:

Element	Details
Customer Segment	High school and college students overwhelmed by complex courses; self-taught programmers; professionals learning new subjects.
Jobs-to-be-Done	Understand tough topics via analogies, query a tutor for clarification, test memory retention with custom quizzes.
Triggers	Approaching exams, failure to understand dense textbook definitions, lack of access to private human tutors.
Emotions (Before/After)	Before: Isolated, overwhelmed, and confused. After: Empowered, confident, and clear on subject fundamentals.
Solution Offered	A unified learning web app combining a customized AI explanation panel, sidebar chatbot tutor, and mock quiz scoring engine.

4.2 Proposed Solution Summary

#	Parameter	Description
1	**Core Value Proposition**	Democratizes tutoring by providing 24/7 custom analogies and interactive grading at zero cost.
2	**Novelty / Uniqueness**	Multiplexes multiple Gemini prompts (explanation, chat context, quiz generation) into a unified FastAPI service.
3	**Scalability**	Stateless API handlers and lightweight Streamlit client allow hosting on elastic containers.

4.3 Solution Architecture

EduGenie implements a modern, layered software design comprising three main components:

• **Presentation Layer (Streamlit Frontend):** Renders the user-facing workspace dashboard, sidebar chat drawer, active quiz panels, and PDF download triggers. It communicates with the backend via RESTful HTTP JSON protocols.
• **Application Layer (FastAPI Backend):** Manages routing, parses incoming payloads, executes local logging, and handles formatting. It provides isolated endpoints (`/api/explain`, `/api/chat`, `/api/quiz/generate`) and serves the automated API documentation page.
• **Integration Layer (Gemini Service):** Translates learning levels and styles into structured system instructions, formats prompts to return stable JSON structures, and establishes conversation history models for chat tutoring.

Component Description Table

Component	Role / Responsibilities	Technologies Used
Web Interface	Captures user inputs, displays custom explanation cards, renders interactive multiple choice quizzes.	Streamlit, HTML, CSS
Backend Routing	Validates request parameters, coordinates service executions, triggers local JSON file updates.	FastAPI, Pydantic, Python
AI Orchestrator	Constructs structured Gemini prompts, handles system instructions, formats raw outputs to valid JSON.	Google Gemini API (GenerativeAI)
Reporting Engine	Generates beautifully formatted A4 study sheets and scorecards dynamically.	ReportLab PDF Library

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was executed in a 6-week sprint model utilizing Agile methodologies. Work was divided into distinct user stories, estimated on a Fibonacci scale (Story Points: 1 = Easy, 8 = Complex). The backlogs below outline the execution path:

Sprint 1 — Workspace Scaffolding & Setup

Story	Task	Points	Priority	Owner
US-1	Setup Streamlit and FastAPI project structure, port routing, and environment variables.	3	High	Manan
US-1	Create base UI templates, sidebar configuration, and layout containers.	3	High	Manan

Sprint 2 — Core Explainer & Gemini Service

Story	Task	Points	Priority	Owner
US-2	Build Gemini service helper. Formulate prompt templates for levels and styles.	5	High	Manan
US-2	Implement FastAPI /explain endpoint, local search logs, and error validators.	5	High	Manan

Sprints (Continued)

Sprint 3 — AI Chat Tutor & Sidebar

Story	Task	Points	Priority	Owner
US-3	Design Streamlit sidebar chat component; bind input text area and load spinners.	5	High	Manan
US-3	Implement FastAPI /chat endpoint with history schemas, passing chat contexts.	5	High	Manan

Sprint 4 — Quiz Generation & Scoring

Story	Task	Points	Priority	Owner
US-4	Build structured JSON schema prompt for Gemini to return clean MCQ arrays.	8	High	Manan
US-4	Develop Streamlit quiz grading panels, score logs, and solution reviews.	8	High	Manan

Sprint Summary & Velocity Table

Sprint	Epic Focus Areas	Total Story Points Delivered
Sprint 1	Scaffolding & Layout Config	6
Sprint 2	Core Explainer Engine Development	10
Sprint 3	Tutor Chat & History Context Routing	10
Sprint 4	Quiz Schema & Performance Grader	16
Sprint 5	ReportLab PDF Exporter Implementation	11
Sprint 6	Pytest Suites & Performance Polish	8

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Functional Testing

To verify system logic, functional testing was carried out across all endpoints and UI operations. The table below lists the test execution cases:

ID	Scenario / Case	Expected Result	Result
FT-01	Custom Topic Explanation	Gemini returns customized markdown; loaded correctly on UI.	Pass
FT-02	Tutor Chat Context	Assistant remembers context and replies correctly to follow-ups.	Pass
FT-03	Quiz MCQ Generation	Stable JSON structure generated with questions and options.	Pass
FT-04	Instant MCQ Grading	Grades answers; displays scores and detailed explanations.	Pass
FT-05	ReportLab PDF Export	PDF scorecard compiled without errors, downloading instantly.	Pass

6.2 Performance Considerations

• **Gemini Model Choice:** Gemini 1.5 Flash was chosen specifically for its fast processing speed. Response times for explanations and quizzes consistently render under 2.5 seconds.
• **Decoupled Architecture:** By offloading AI prompts and file logs to the FastAPI backend, the Streamlit UI remains responsive and prevents blocking during processing events.
• **Asynchronous Handlers:** Async request processing in the backend ensures multiple active clients can interact with the assistant simultaneously without queuing delays.

7. RESULTS

7.1 Output Screenshots & Milestones

All planned features were successfully developed, tested, and demonstrated. The following grid maps out the functional milestones achieved:

Feature Area	Status	Demonstrated	Remarks / Implementation Details
Topic Explanation Panel	Completed	Yes	Provides custom formatting based on user learning parameters.
AI Tutor Chat Drawer	Completed	Yes	Sidebar chat drawer keeps local memory logs for follow-up.
Interactive MCQ Quizzes	Completed	Yes	MCQ arrays structured via custom JSON schema formats.
Performance PDF Export	Completed	Yes	Generates clean, printable A4 summaries dynamically.
History Scoreboards	Completed	Yes	Saves previous topics and scorecard data locally to JSON.

7.2 Code Quality & Reusability

Aspect Evaluated	Rating (1-5)	Comments & Design Rationale
Code Layout & Structure	4 / 5	Clear modularity: separate folder structures for frontend, backend API routers, services, and tests.
Readability & Types	4 / 5	Strict typing schema setups in schemas.py prevent format errors.

Code Quality (Continued)

Aspect Evaluated	Rating (1-5)	Comments & Design Rationale
Reusability of Loggers	4 / 5	Orchestration logic in gemini_service can be reused across different frontend interfaces without rewrite.
Documentation & Comments	3 / 5	Docstrings explain core router logic, though inline comments could be expanded.
Overall Project Score	4 / 5	System meets all user goals, runs locally without latency, and is robust for academic submission.

Design Best Practices Implemented

• **Input Schema Isolation:** By defining schemas using Pydantic, the backend automatically validates payload types before running prompt engines, eliminating parameter injection errors.
• **API Isolation:** The presentation layer cannot access API keys or system prompt templates directly. This enforces a strict separation of concerns and prevents configuration leakage.
• **Clean Waterfalls:** Browser network calls are kept synchronous for critical operations (like generating explanations), but dashboard widgets and sidebar loaders run asynchronously to improve responsiveness.

8. ADVANTAGES & DISADVANTAGES

Key Advantages

• **Personalized Explanations:** Provides highly customized, analogy-based learning that accommodates beginner, intermediate, and advanced levels, unlike generic static textbooks.
• **Zero Operating Cost:** Uses Gemini's large free tier combined with local storage logs, avoiding database fees during development.
• **Instant Knowledge Testing:** Generates and grades quizzes immediately, providing solutions and detailed explanations for incorrect options.
• **Decoupled REST backend:** FastAPI router is easy to maintain and can scale independently of the Streamlit client.
• **Document Exporter:** Programmatic A4 PDF generator allows students to print or share scorecards easily.

Key Disadvantages & Limitations

• **API Dependency:** App requires internet access and active Gemini API uptime to generate explanations and quizzes.
• **No Multi-User Accounts:** Uses local JSON logs for history, which are shared across runs unless user-specific logins are established.
• **Manual Progress Tracking:** Scores are tracked locally; students cannot synchronize progress across different browsers or devices.

9. CONCLUSION

EduGenie: Google Gemini-Powered Learning Assistant successfully demonstrates how generative AI and modular API routing can be combined to resolve educational bottlenecks. Across six sprints, the application evolved into a stable, secure, and user-centric portal that replaces static reading materials with interactive explanations and instant testing feedback loop.

The project backend routes requests cleanly across the custom explainer engine, conversational tutor, and MCQ quiz generator using FastAPI, while Streamlit serves as a lightweight, highly responsive client interface. Functional verification confirmed that all core modules operate reliably and return the expected results under normal network loads.

While enhancements like multi-user cloud databases and speech-to-text integration remain in the roadmap, the current local implementation stands as a highly polished, fully working prototype that proves the viability of AI-assisted personalized study environments.

10. FUTURE SCOPE

A phased roadmap has been designed to transition the application from a local learning prototype into a production-grade, multi-user educational cloud platform:

Phase	Planned Enhancement Details	Timeline	Expected User Impact
Phase 2	Integrate user login authentication and cloud database persistence (Supabase/PostgreSQL).	3 months	Secures study histories, enabling progress tracking across multiple devices.
Phase 3	Incorporate speech-to-text and voice synthesizers inside AI Tutor Chat.	3-6 months	Enables hands-free study, assisting visually impaired students.
Phase 4	Incorporate school curriculum mappings and collaborative study rooms.	6-9 months	Enables group discussions, mock tests, and teacher monitoring panels.
Phase 5	Launch native Android and iOS mobile application wrapper.	9-12 months	Provides offline caching and mobile-optimized interfaces.

11. APPENDIX

Source Code Overview

The project source code is organized modularly to decouple presentation, business logic, and QA testing layers. Below is the file directory layout representing the active codebase:

```
EduGenie-Google-Gemini-Powered-Learning-Assistant/
|
|-- app/                    # FastAPI API Routing Tier
|   |-- main.py             # API Entrypoint Router
|   |-- config.py           # Key & Host Settings
|   |-- models/             # Input/Output Schemas
|   |   L-- schemas.py
|   |-- routers/            # Endpoint Handler Routers
|   |   L-- learning.py     # explain/chat/quiz logic
|   L-- services/           # Gemini API Service Helper
|       |-- gemini_service.py
|       |-- history_logger.py
|       L-- pdf_generator.py
|
|-- frontend/               # Presentation Tier UI
|   |-- streamlit_app.py    # Web UI Interface
|   L-- assets/             # Watermarks & Graphics
|       |-- index.html      # HTML Layout wrapper
|       L-- edugenie_logo.png
|
|-- tests/                  # Quality Assurance Tier
|   L-- test_backend.py     # Pytest router test suite
|
|-- package.json            # Project Metadata configs
|-- run_backend.bat         # Backend Startup Script
L-- run_frontend.bat        # Frontend Startup Script
```

Project Metadata Configuration (package.json)

The package.json file defines metadata, scripts, and runtime dependencies for the learning assistant:

```
{
  "name": "edugenie-learning-assistant",
  "version": "1.0.0",
  "description": "Google Gemini-Powered Learning Assistant",
  "main": "app/main.py",
  "dependencies": {
    "fastapi": "^0.110.0",
    "uvicorn": "^0.27.1",
    "streamlit": "^1.31.1",
    "google-generativeai": "^0.4.0",
    "reportlab": "^4.1.0",
    "pydantic": "^2.6.1",
    "pytest": "^8.0.0"
  },
  "author": "Manan Kochar",
  "license": "MIT"
}
```

FastAPI Routing Gateway (app/main.py Excerpt)

```
from fastapi import FastAPI
from app.routers.learning import router as learning_router
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="EduGenie Learning Assistant API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(learning_router)

@app.get("/")
def root():
    return {"status": "online", "service": "EduGenie API"}
```

API Router Handlers (app/routers/learning.py Excerpt)

The endpoint handlers parse parameter models and call the Gemini Service helper:

```
from fastapi import APIRouter, HTTPException
from app.models.schemas import ExplainRequest, ExplainResponse
from app.services import gemini_service

router = APIRouter(prefix="/api")

@router.post("/explain", response_model=ExplainResponse)
def explain(request: ExplainRequest):
    try:
        explanation_data = gemini_service.explain_topic(
            topic=request.topic,
            level=request.level,
            style=request.style
        )
        return explanation_data
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Pydantic Validation Schemas (app/models/schemas.py)

```
from pydantic import BaseModel
from typing import Optional, List

class ExplainRequest(BaseModel):
    topic: str
    level: str
    style: str

class ExplainResponse(BaseModel):
    topic: str
    explanation: str
    analogies: Optional[List[str]] = []
    summary: Optional[str] = None
```

Dataset Link

This project does not consume any pre-existing external static dataset. All academic topic explanations, conversation starters, and quiz multiple-choice questions are generated in real-time by the Google Gemini 1.5 Flash model based on current user inputs, and cached locally inside history.json storage files.

GitHub & Project Demo Link

- **GitHub Repository:**

https://github.com/manankochar1720/Gen-ai-Google-Gemini-Powered-Learning-Assistant

- **Project Demo Video:**
https://drive.google.com/file/d/1bKsPvajDTE8vF-AaRfxUXIssAQp171tJ/view?usp=sharing

Name	Project Role / Responsibilities
Manan Kochar	Team Lead — Full-Stack Developer (FastAPI Backend API setup, Streamlit Frontend interface design, Google Gemini AI prompt orchestration, and ReportLab PDF Report compiler)